RIS Lab II
Report for Assignment 3

Yiheng Cao, Eduard-Antonio Zippenfenig, Wai Huen Sin, Amanuel Basaznew Legesse

***Abstract*— In this assignment, We Recap the Vocabulary learned within the ROS system, Change the pattern of LED lights on the Duckietown and Control the Duckietown automatically when it approaches the red lines.**

TASK 1 KNOW YOUR WAY AROUND

a) typical folder structure of a ROS workspace:

```
catkin_ws/
├── src/
├── build/
└── devel/
```

catkin_ws - Root folder of the workspace. The place where we run catkin build.

src/ - Source space that stores all ROS packages by catkin build.

build/ - Contains temporary build files generated during catkin build.

devel/ - Contain built packages and development environments files (setup, executable, etc)

b) main folders and files

```
catkin_ws/
└── src/
    └── package/
        ├── package.xml
        ├── CMakeLists.txt
        ├── src/
        │   └── node.cpp
        └── launch/
```

package.xml - defines package name, version, license, maintainer, and dependency

CMakeLists.txt - Defines how the packages are built (executables, libraries, dependencies).

setup.bash - it adds workspace to your ROS environment (source devel/setup.bash)

ROS package - contains nodes, configuration files, and provides a ROS interface.

The catkin packages must include:

- a package.xml file and a CMakeLists.txt
- Each package must have its own folder

git repository - used to track changes and collaborate on ROS packages or workspaces with others.

c) How does the "catkin build" command interact with these files?

When running catkin build:

1) scans the src/ directory for packages
2) reads package.xml to resolve dependencies.
3) runs CMakeLists.txt to know how to compile the package.
4) compiles source code in src/.
5) generates executables, libraries, and messages.
6) Store results in build/ and devel/.

d) Standard ways within ROS to pass information to the node on startup or at runtime.

- Using rosrun, run a single node directly from a package
  `$ rosrun [package_name] [node name]`
- Using launch files, XML files (.launch) located in the launch/
  `$ roslaunch [package_name] [launchfile.launch]`
  They allow starting multiple nodes and configuring them
  Enables full configuration:
  - Which nodes are run
  - What parameters they use
  - Which topics they communicate on
- Using parameters, stored on the ROS Parameter Server
  Example of ROS parameter configuration:

```
<node pkg="turtlesim" type="
    turtlesim_node" name="sim">
    <param name="background_r" value="200
        " type="int"/>
</node>
```

To set this via command line, execute:

```
$ rosparam set /sim
```

They allow passing configuration values to nodes.

rostopic/rosnode: rostopic/rosnode are used to provide a list of all the available topics/nodes. In our case it was useful when parsing through the list of topics to search for joystick_override and determining how to move the duckiebot with a ros command directly and not the gui.

rosparam: rosparam was used to give an overview of what parameters where part of the kinematics node. Especially for understanding that it consisted of tangential and angular components alongside the trim variable, which we would examine and graph for the second report.

rosbag: rosbag was used for recording published topics. It was used when observing the differences between the topics and published messages directly from the duckiebot versus the one from the laptop.

rqt_graph: rqt_graph is a tool for observing a visual representation of how the topics that are published relate to one another. It gave us an overview of relevant topics that are responsible for certain tasks like controlling the virtual joystick, lane following, and the like in our case.

Rviz: Rviz is what provided a virtual representation of the duckiebot and its surroundings. It allows us to visualize topics such as the markerArrays for the lane_pose visualizer and the line_segment visualizer nodes.

plotjuggler: Plotjuggler provides sensor data over time that we have recorded from the rosbags. It was useful for showing the published twist messages over time of the duckiebot.

One way of trying to solve it would be to see which relevant topics are linked in Rviz. Then go to the rqt_graph to better see what nodes are connected to the sensor data. We could then potentially try to use rostopic/rosnode list to see what might be causing the issue similarly to how the joystick_override command worked in the third report. After looking through the relevant nodes, we can then maybe see if there are any parameters that could be adjusted like the trim value, velocity or something similar. Another thing to do is check with plotjuggler if the topics are publishing messages correctly.

Task 2 Mood Lights

```
eduard@cu:~/catkin_ws$ rostopic info /teal/led_emitter_node/led_pattern
Type: duckietown_msgs/LEDPattern

Publishers:
 * /teal/led_emitter_node (http://teal.local:35787/)

Subscribers:
 * /teal/led_driver_node (http://teal.local:38547/)


eduard@cu:~/catkin_ws$ rosmsg info duckietown_msgs/LEDPattern
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
string[] color_list
std_msgs/ColorRGBA[] rgb_vals
  float32 r
  float32 g
  float32 b
  float32 a
int8[] color_mask
float32 frequency
int8[] frequency_mask
```

Fig. 1. The message information on the topic duckietown_msgs/LedPattern

Fig. 2. robot with RED front and BLUE rear

Fig. 3. robot with BLUE front and RED front

Task 3 Stop and Go

```
qwe@qwe-virtual-machine:~$ rosmsg show duckietown_msgs/Segment
uint8 WHITE=0
uint8 YELLOW=1
uint8 RED=2
uint8 color
duckietown_msgs/Vector2D[2] pixels_normalized
  float32 x
  float32 y
duckietown_msgs/Vector2D normal
  float32 x
  float32 y
geometry_msgs/Point[2] points
  float64 x
  float64 y
  float64 z
```

Fig. 4. detailed information of Segment message types

```
qwe@qwe-virtual-machine:~$ rosmsg show duckietown_msgs/SegmentList
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
duckietown_msgs/Segment[] segments
  uint8 WHITE=0
  uint8 YELLOW=1
  uint8 RED=2
  uint8 color
  duckietown_msgs/Vector2D[2] pixels_normalized
    float32 x
    float32 y
  duckietown_msgs/Vector2D normal
    float32 x
    float32 y
  geometry_msgs/Point[2] points
    float64 x
    float64 y
    float64 z
```

Fig. 5. detailed information of SegmentList message types

After building the stop_and_go package, launch the virtual joysticker to active the autonomous driving. When the duckietown approach to the red lines, it will stop and change the front two LEDs into red and pause for seconds. After that, it will continue driving autonomously.

Contributions

- Yiheng Cao - Complete Task 3
- Eduard-Antonio Zippenfenig - Complete Task 2 and 3
- Wai Huen Sin - Complete Task 1 and 2
- Amanuel Basaznew Legesse - Complete Task 1 and 3